Monte Carlo Pi Estimation - Multithreading Lab

Java, native threads, race conditions, synchronization, reduction

Test machine: MacBook Air, 8 logical cores, 8 physical cores.

Part 1: The Phantom Bug

50,000,000 random points were generated and split across 4 native Java threads. All threads updated a shared variable totalHits with totalHits++ and no synchronization. The program was run 5 times. Results:

Run	Hits	Estimated Pi	Time (ms)
1	13,743,558	1.0995	906.3
2	15,115,340	1.2092	781.6
3	15,794,535	1.2636	768.2
4	15,307,211	1.2246	770.5
5	16,024,226	1.2819	764.7

The result is far from the correct value of Pi (3.1415...) and it changes on every run. This happens because totalHits++ is not a single atomic operation. It is really three steps: read the value, add one, write the value back. When 4 threads perform these steps at the same time, some increments get overwritten and lost. This is a classic data race, and it explains why the measured Pi values are unstable and consistently too low.

Part 2: The Synchronization Trap

The race condition was fixed in two ways: wrapping totalHits++ in a synchronized block, and replacing the shared variable with an AtomicLong using incrementAndGet(). Both were compared against a plain single threaded loop, all using 50,000,000 points.

Version	Estimated Pi	Time (ms)
Synchronized (4 threads)	3.1417	2313.3
AtomicLong (4 threads)	3.1421	604.0
Single thread (no locks)	3.1414	172.1

Both fixes give an accurate result, close to 3.1415. However the synchronized version took about 2313 ms, which is over 13 times slower than the single threaded version, even though it uses 4 threads. The AtomicLong version is also slower than the single thread version, but far less slower than the synchronized version. This shows that correctness fixes can come with a large performance cost if they force threads to constantly compete for the same lock.

Part 3: OpenMP Style Reduction

Each thread was given a private local counter instead of touching any shared variable during the loop. Partial sums were combined only once, after all threads finished (join). This mirrors the OpenMP pattern reduction(+:totalHits). The benchmark used 100,000,000 points for each thread count T.

Threads (T)	Runtime (ms)	Speedup (T1 / TN)	Efficiency (Speedup / T)
1 (baseline)	358.3	1.00x	100.0%
2	193.5	1.85x	92.6%
4	109.6	3.27x	81.7%
8	106.1	3.38x	42.2%
16	116.7	3.07x	19.2%
32	168.2	2.13x	6.7%

Questions

1. Look at your row for 16 threads. Why didn't your 8 core CPU run twice as fast as 8 threads?

The test machine has 8 logical cores and 8 physical cores, with no extra hardware threads per core. Once the thread count goes above 8, there are more threads than cores available to run them. The operating system has to schedule multiple threads on the same core and switch between them, which adds context switching overhead and does not add any real parallel processing power. This is why runtime for 16 threads (116.7 ms) is actually worse than for 8 threads (106.1 ms), and speedup drops from 3.38x to 3.07x. The CPU is oversubscribed, not doubled.

2. Why was the synchronized version in Part 2 slower than running on one single core?

The synchronized block forces every single increment to acquire and release a lock before it can run. Even though 4 threads are working, only one thread can hold the lock and increment the counter at any given moment, so the actual increment work becomes serialized again, just like a single thread. On top of that, threads that are waiting for the lock get blocked and must be woken up later by the operating system, and every lock acquisition also forces a memory barrier that keeps CPU caches in sync across cores. Since the point calculation in each loop iteration is cheap, almost all of the time ends up being spent on lock contention and cache coherency traffic instead of useful work, which makes the synchronized version slower than a plain single threaded loop with no locking at all.

Conclusion

Unprotected shared state gives fast but wrong results (Part 1). Protecting shared state with locks or atomics gives correct results but adds heavy overhead from contention (Part 2). Giving each thread its own private counter and combining results only once at the end gives both correct results and

good performance, up to the point where the number of threads exceeds the number of available CPU cores (Part 3).